# Rapport de projet Opération Research – Master :

# FLOTS ET COUTS

Étudiant : ALECU Luca

Encadrant : Jean Charles Régin

Master Informatique

Université Côte d'Azur

DS4H Projet de production– Semestre 2, 2025–2026

## Table des matières

1. Introduction : 3
1.1 Explication et intérêt du sujet : 3
1.2 Objectifs du projet : 3
2. Structures de données et Classe : 3
2.1 Classe Graph : 3
2.2 Classe Arrête : 4
2.3 Classe MaxFlowResult : 4
2.4 Classe PathResult : 4
2.5 Classe GraphVizWriter : 5
2.6 Classe Nœud : 5
2.7 Classe ParsedGraph : 5
2.8 Classe BoundedEdge, BoundedEdgeRef et LowerBoundResult: 5
2.9 Classe Main: 5
3. Explications des algorithmes : 6
3.1 Ford-Fulkerson / Edmonds-Karp (Flot maximal + Min-cut): 6
3.2 Min-cost flow par chemins augmentants (Bellman-Ford): 6
3.3 Min-cost flow par chemins augmentants (Dijkstra + potentiels): 7
3.4 Détection de cycles négatifs: 7
3.5 Assignation min-cost et contraintes: 8
3.6 Arc vital pour le flot max: 8
4. Conclusion : 8

# 1. Introduction :

## 1.1 Explication et intérêt du sujet :

La recherche opérationnelle occupe une place centrale dans la modélisation et l'optimisation de systèmes complexes. Parmi les problèmes classiques, le flot maximal, le flot de cout minimal et la détection de cycles négatifs constituent des outils fondamentaux pour analyser les réseaux (logistiques, télécommunication, production, énergie).
L'objectif de ce projet est de proposer une implémentation complète et exploitable de ces mécanismes sur des graphes orientes, en respectant les contraintes pédagogiques (pas de bibliothèques de flots) et en fournissant des sorties lisibles et reproductibles via GraphViz.

Le logiciel développé s'articule autour d'un graphe résiduel et d'algorithmes robustes: Ford-Fulkerson (version Edmonds-Karp), min-cost flow par chemins augmentant avec deux stratégies de plus court chemin (Bellman-Ford et Dijkstra avec potentiels), ainsi qu'une routine de détection de cycles négatifs. Une attention particulière est portée à la qualité de l'ingénierie logicielle: séparation claire des classes, structures de données explicites, et génération automatique de fichiers .gv représentant les solutions.
Ce projet à été réalisé a l'aide du langage de programmation JAVA.

## 1.2 Objectifs du projet :

Les objectifs peuvent être résumés en quatre axes:
1) Implémenter le flot maximal et la min-cut sur un graphe oriente avec capacités.
2) Implémenter le flot de cout minimal via des chemins augmentant, en deux variantes:
    -Bellman-Ford (couts négatifs) et Dijkstra avec renormalisation des couts.
3) Détecter les cycles de cout négatif dans le graphe résiduel.
4) Produire des sorties visuelles (GraphViz) et des exemples conformes au sujet.

# 2. Structures de données et Classe :

La conception s'appuie sur un graphe résiduel, avec une représentation explicite des arcs et des flux. Les classes ont été séparées pour clarifier les responsabilités et faciliter la lecture.
Les sections suivantes expliquent chaque classe et son utilité.

## 2.1 Classe Graph :

La classe Graph constitue la structure centrale du programme : elle représente un graphe orienté résiduel et regroupe toutes les opérations de calcul, notamment le flot maximal, le flot à coût minimal, la coupe minimale et la détection de cycles. Elle contient le nombre de nœuds n, un tableau adj composé des listes d'arcs sortants de chaque nœud (liste d'adjacence), une liste edges regroupant les arcs originaux sans les arcs inverses résiduels, ainsi qu'un compteur nextEdgeId permettant d'attribuer un identifiant unique à chaque arc original. La méthode addEdge crée à la fois un arc direct, doté d'une capacité positive et d'un coût associé, et un arc inverse résiduel de capacité nulle et de coût opposé. Ces deux arcs sont reliés entre eux grâce à un pointeur rev, ce qui permet de propager automatiquement le flot et d'effectuer des annulations dans le graphe résiduel.

La classe Graph encapsule l'algorithme d'Edmonds-Karp (BFS sur le résiduel) pour le flot maximal et calcule ensuite la min-cut en marquant les sommets atteignables depuis la source dans le résiduel. Pour le flot de cout minimal, Graph implémente deux variantes de chemins augmentant: Bellman-Ford (robuste a des couts négatifs) et Dijkstra avec potentiels (plus rapide, couts réduits). La détection de cycles négatifs repose sur un Bellman-Ford applique au résiduel.

## 2.2 Classe Arrête :

La classe Arrête représente un arc dans le graphe résiduel et contient plusieurs attributs essentiels : from et to, qui désignent respectivement le sommet source et le sommet destination, id, un identifiant unique attribué uniquement aux arcs originaux, original, un booléen indiquant si l'arc est un arc original ou un arc résiduel, capacity, correspondant à la capacité maximale de l'arc, cost, qui représente le coût unitaire, flow, indiquant le flot actuel, ainsi que rev, une référence vers l'arc inverse. La méthode residualCapacity renvoie la valeur capacity - flow, c'est-à-dire la capacité encore disponible sur l'arc. Le champ rev est fondamental pour les algorithmes de flot : lorsqu'on augmente le flot sur un arc direct, cela revient à diminuer le flot sur l'arc inverse, ce qui permet de modifier le graphe résiduel de manière cohérente tout en assurant la propagation correcte du flot et des possibilités d'annulation.

## 2.3 Classe MaxFlowResult :

La classe MaxFlowResult est un conteneur de résultats pour le calcul du flot maximal. Elle stocke maxFlow, qui représente la valeur totale du flot, minCutCapacity, correspondant à la capacité totale de la coupe minimale, ainsi que minCutEdges, la liste des arcs appartenant à cette coupe. Cette classe permet d'isoler les résultats et de clarifier l'interface du programme, tout en offrant à Main la possibilité de produire des sorties détaillées sans exposer la logique interne des algorithmes. La classe MinCostFlowResult constitue quant à elle un conteneur de résultats pour le calcul du flot de coût minimal. Elle contient flow, représentant le flot total acheminé, et cost, correspondant au coût total associé. Le coût est stocké dans un type long afin de limiter les risques de débordement lors du traitement de graphes plus denses.

## 2.4 Classe PathResult :

La classe PathResult est une structure interne utilitaire utilisée pour les calculs de plus court chemin. Elle contient dist, qui représente les distances depuis la source, correspondant aux coûts réduits dans le cas de l'algorithme de Dijkstra, prev, un tableau d'arcs précédents permettant de reconstruire un chemin, ainsi que found, un booléen indiquant si le puits a été atteint. Cette structure permet d'isoler la logique liée au calcul de plus court chemin afin de faciliter sa réutilisation dans les deux variantes de l'algorithme de flot à coût minimal.

## 2.5 Classe GraphVizWriter :

La classe GraphVizWriter permet d'exporter un graphe ainsi que ses flux vers un fichier .gv. Elle écrit les arcs originaux en leur associant un label HTML indiquant le flot et la capacité en vert, ainsi que le coût en rouge. Elle écrit également les nœuds avec un label numérique et applique des couleurs spécifiques à la source et au puits. Cette classe est utilisée pour la visualisation des graphes et la production de rendus PDF à l'aide de GraphViz, tout en assurant la conformité aux consignes de l'enseignement.

## 2.6 Classe Nœud :

Nœud représente un nœud avec un id et un label. Elle est utile pour des étiquettes lisibles dans des extensions futures. Elle n'est pas indispensable aux algorithmes, mais clarifie l'intention de la modélisation.

## 2.7 Classe ParsedGraph :

ParsedGraph regroupe le graphe, la source et le puits lus depuis un fichier d'entrée. Cette classe simplifie le passage des informations entre la fonction de parsing et les fonctions d'exécution.

## 2.8 Classe BoundedEdge, BoundedEdgeRef et LowerBoundResult:

BoundedEdge représente une arrête avec borne inferieure (lower) et borne supérieure (capacity). Elle est utilisée pour l'assignation avec contrainte "chaque tache prise au moins une fois", ou l'on impose un flot minimal sur les arcs tache->puits.

BoundedEdgeRef conserve un lien entre une arrête d'origine (avec borne) et l'arrête transformée du graphe résiduel. Elle permet de reconstruire le flot réel après transformation des bornes inferieures.

La classe LowerBoundResult encapsule le résultat de la transformation des bornes inférieures. Elle contient graph, représentant le graphe transformé intégrant une super-source et un super-puits, superSource et superSink, qui correspondent aux nœuds ajoutés lors de cette transformation, totalDemand, représentant la demande totale à satisfaire, refs, qui regroupe les références vers les arcs transformés, ainsi que baseCost, correspondant au coût constant induit par les bornes inférieures. Cette structure formalise un schéma classique utilisé dans les algorithmes de flot à coût minimal : la réduction d'un problème comportant des bornes inférieures vers un problème standard de flot.

## 2.9 Classe Main:

La classe Main orchestre l'exécution du programme. Elle gère les arguments de ligne de commande, le chargement des fichiers d'entrée, les exemples intégrés tels que le flot maximal, le problème d'affectation, le flot à coût minimal et l'arc vital, ainsi que la génération des fichiers .gv. Elle contient également les méthodes permettant de construire les graphes d'exemple et d'interpréter le format des fichiers d'entrée.

## 3. Explications des algorithmes :

### 3.1 Ford-Fulkerson / Edmonds-Karp (Flot maximal + Min-cut):

Dans le cadre de ce projet, l'objectif est de calculer le flot maximal entre une source s et un puits t, puis d'en déduire une coupe minimale associée. Dans un graphe capacitaire, le théorème max-flow min-cut établit que la valeur du flot maximal est égale à celle de la coupe minimale. Le principe de l'algorithme repose sur l'exploitation du graphe résiduel : tant qu'il existe un chemin de s vers t disposant d'une capacité résiduelle strictement positive, on augmente le flot le long de ce chemin d'une valeur égale au goulot d'étranglement, c'est-à-dire la capacité résiduelle minimale rencontrée sur ce chemin. Après chaque augmentation, les flots sont mis à jour, ainsi que les arcs inverses, afin de maintenir une représentation cohérente du résiduel.

L'implémentation retenue repose sur l'algorithme d'Edmonds-Karp, dans lequel les chemins augmentant sont déterminés par un parcours en largeur (BFS), garantissant la sélection des plus courts chemins en nombre d'arcs. Cette stratégie assure une complexité polynomiale en O(V * E^2), adaptée à une mise en œuvre pédagogique et robuste. Une fois l'algorithme convergé, la coupe minimale est obtenue en identifiant l'ensemble des sommets atteignables depuis s dans le graphe résiduel. Les arcs originaux reliant un sommet atteignable à un sommet non atteignable constituent alors la coupe minimale, et la somme de leurs capacités correspond à la valeur de cette coupe, égale au flot maximal.

Cette approche présente plusieurs avantages dans un cadre de niveau master : elle garantit la terminaison et une complexité polynomiale via Edmonds-Karp, elle reste conceptuellement simple à implémenter et permet d'obtenir simultanément le flot maximal et la coupe minimale dans une même exécution. En revanche, elle devient moins efficace sur de grands graphes comparativement à des algorithmes plus avancés tels que Dinic ou Push-Relabel, notamment en raison de la répétition d'un BFS à chaque itération augmentante.

### 3.2 Min-cost flow par chemins augmentants (Bellman-Ford):

L'objectif est d'acheminer un flot maximal possible entre une source s et un puits t tout en minimisant le coût total, défini comme la somme des produits entre les coûts unitaires des arcs et les flots qui les traversent. Le principe consiste, à chaque itération, à rechercher dans le graphe résiduel un chemin de s vers t de coût minimal en tenant compte des coûts associés aux arcs, puis à augmenter le flot sur ce chemin d'une quantité égale au goulot résiduel. Ce processus est répété jusqu'à ce qu'il n'existe plus de chemin améliorant, c'est-à-dire jusqu'au blocage de l'algorithme.

Le choix de l'algorithme de Bellman-Ford s'explique par sa capacité à gérer des coûts négatifs, qui apparaissent naturellement dans le graphe résiduel en raison des arcs inverses. Il permet également la détection de cycles de coût négatif, ce qui est essentiel pour garantir la validité d'une solution dans un cadre général de flot à coût minimal.

Cette approche présente plusieurs avantages : elle supporte les coûts négatifs, elle reste simple et fiable à mettre en œuvre, et elle permet la détection explicite des cycles négatifs. En revanche, sa complexité en O(V * E) par itération peut entraîner des performances limitées, notamment sur des graphes denses ou de grande taille, ce qui peut la rendre moins efficace que des variantes plus optimisées.

### 3.3 Min-cost flow par chemins augmentants (Dijkstra + potentiels):

L'objectif est de résoudre le même problème que précédemment, à savoir acheminer un flot maximal entre une source s et un puits t en minimisant le coût total, mais en utilisant une approche plus efficace en pratique. Le principe repose sur l'utilisation de potentiels (π) permettant de renormaliser les coûts des arcs selon la formule $c'(u,v) = c(u,v) + \pi[u] - \pi[v]$. Lorsque les potentiels sont correctement définis, les coûts réduits deviennent non négatifs, ce qui rend possible l'utilisation efficace de l'algorithme de Dijkstra pour la recherche des plus courts chemins dans le graphe résiduel. À chaque itération, un chemin de coût minimal est trouvé avec ces coûts réduits, puis le flot est augmenté sur ce chemin en fonction du goulot résiduel.

L'initialisation des potentiels est réalisée à l'aide d'un algorithme de Bellman-Ford exécuté depuis la source, afin de gérer correctement les coûts initiaux, y compris les coûts négatifs. Par la suite, les potentiels sont mis à jour à chaque itération en fonction des distances calculées, ce qui garantit la validité des coûts réduits pour l'application de Dijkstra.

Cette approche présente plusieurs avantages : elle est généralement plus rapide en pratique que l'algorithme basé uniquement sur Bellman-Ford, et elle est adaptée aux grands graphes lorsque les coûts restent relativement stables. En revanche, elle nécessite une phase d'initialisation par Bellman-Ford ainsi qu'un suivi rigoureux des potentiels, ce qui rend l'implémentation plus délicate. Elle peut également être sensible aux erreurs numériques lorsque les coûts sont très grands, ce qui impose une attention particulière au choix des types de données.

### 3.4 Détection de cycles négatifs:

L'objectif est de vérifier l'absence de cycle de coût négatif dans le graphe résiduel, car la présence d'un tel cycle rend le problème de flot à coût minimal mal posé, dans la mesure où il serait possible de réduire indéfiniment le coût total en circulant dans ce cycle. Le principe repose sur l'application de l'algorithme de Bellman-Ford en considérant tous les sommets avec une distance initiale nulle (ou en assimilant cette configuration à l'utilisation d'un super-source implicite). Si, après V-1 itérations de relaxation, il est encore possible d'améliorer une distance, cela implique l'existence d'un cycle de coût négatif dans le graphe.

Cette méthode présente plusieurs avantages : il s'agit d'une approche classique, robuste et fiable, permettant en outre une vérification explicite de la validité des données du problème. En revanche, elle introduit un coût computationnel supplémentaire en O(V * E), ce qui peut être significatif dans des contextes de grande taille ou lorsqu'elle est appliquée fréquemment dans une chaîne de calculs.

### 3.5 Assignation min-cost et contraintes:

Le problème d'assignation est modélisé sous forme de flot dans un réseau : une source est reliée aux personnes avec une capacité de 1, les personnes sont reliées aux tâches avec une capacité de 1 et des coûts associés, et les tâches sont reliées au puits avec des capacités dépendant du cas considéré. Afin de satisfaire la contrainte selon laquelle chaque tâche doit être assignée au moins une fois, des bornes inférieures sont introduites sur les arcs allant des tâches vers le puits, ce qui nécessite une transformation du problème par l'ajout d'une super-source et d'un super-puits. Le coût de base induit par ces bornes inférieures est ensuite intégré au coût final de la solution.

Cette formulation correspond à une approche classique en optimisation linéaire sur réseaux. Elle permet de ramener un problème avec contraintes supplémentaires vers un problème standard de flot à coût minimal, tout en conservant l'utilisation d'un solveur générique de min-cost flow.

### 3.6 Arc vital pour le flot max:

L'arc vital est défini comme celui dont la suppression diminue le plus le flot maximal. L'approche retenue consiste à recalculer le flot maximal pour chaque arc supprime. C'est une méthode exhaustive, simple, mais couteuse (O(E) calculs de flot). Elle est acceptable pour un contexte pédagogique et de petite taille.

## 4. Conclusion :

Ce projet fournit une implémentation complète et pédagogique des algorithmes de flot maximal, de min-cut et de flot de cout minimal, ainsi qu'une détection de cycles négatifs. Les choix d'implémentation privilégient la clarté et la robustesse, tout en restant efficaces pour des graphes de taille modérée. La séparation des structures de données, la génération automatique de GraphViz et la présence d'exemples couvrant les cas du sujet rendent le projet exploitable pour l'apprentissage et l'évaluation.

Les limites identifiées (performances pour grands graphes, approche exhaustive pour l'arc vital) peuvent constituer des pistes d'amélioration. Toutefois, dans le cadre d'un projet de niveau master, l'architecture et les algorithmes proposes offrent un compromis pertinent entre rigueur, transparence et conformité aux consignes.